

Монгол Улсын Их Сургууль
Мэдээлэл Технологи Электроникийн Сургууль



Алгоритмын шинжилгээ, зохиомж II (ISCI254)

Санамсаргүй алгоритм

Гүйцэтгэсэн: Э.Бэлэгдэмбэрэл (21B1NUM2890)

2025 он 05 сарын 05

QuickSelect алгоритм болон Randomized BST, Skip List өгөгдлийн бүтцүүдийг хэрэгжүүлж, практикт хэрхэн ажиллаж буйг туршина.

1. QuickSelect алгоритмын ажиллах хурдыг $n = \{10M, 20M, 40M, 80M, 160M\}$ хэмжээтэй санамсаргүй оролт дээр хэмжиж ямар нэг эрэмбэлэх алгоритмтай (жишээ Quicksort) хурдыг нь харьцуулсан график эсвэл хүснэгт байгуул. Ингэхдээ QuickSelect алгоритмийн нэг оролт дээр K -г санамсаргүйгээр 1000 удаа сонгон ажиллуулж дунджаар нь ажиллах хугацааг тооцно.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int quickselect(int arr[], size_t n, size_t k) {
    if (n == 1) return arr[0];
    int pivot = arr[rand() % n];
    int *lows = malloc(n * sizeof(int));
    int *highs = malloc(n * sizeof(int));
    int *pivots = malloc(n * sizeof(int));
    size_t low_c = 0, high_c = 0, pivot_c = 0;

    for (size_t i = 0; i < n; i++) {
        if (arr[i] < pivot) lows[low_c++] = arr[i];
        else if (arr[i] > pivot) highs[high_c++] = arr[i];
        else pivots[pivot_c++] = arr[i];
    }

    int result;
    if (k < low_c)
        result = quickselect(lows, low_c, k);
    else if (k < low_c + pivot_c)
        result = pivot;
    else
        result = quickselect(highs, high_c, k - low_c - pivot_c);

    free(lows);
    free(highs);
    free(pivots);
    return result;
}
```

```

int cmpfunc(const void *a, const void *b) {
    return (*(int*)a - *(int*)b);
}

void benchmark(size_t n) {
    int *data = malloc(n * sizeof(int));
    if (!data) {
        printf("Memory allocation failed for size %zu\n", n);
        return;
    }

    for (size_t i = 0; i < n; ++i) data[i] = rand() % 1000000;

    double total_qs_time = 0;
    for (int i = 0; i < 1000; ++i) {
        int *sample = malloc(n * sizeof(int));
        if (!sample) {
            printf("Memory allocation failed during QuickSelect sample\n");
            break;
        }
        for (size_t j = 0; j < n; ++j) sample[j] = data[j];
        size_t k = rand() % n;

        clock_t start = clock();
        quickselect(sample, n, k);
        clock_t end = clock();
        total_qs_time += (double)(end - start) / CLOCKS_PER_SEC;

        free(sample);
    }

    printf("n=%zu | QuickSelect avg: %.6f s", n, total_qs_time / 1000);

    int *sort_sample = malloc(n * sizeof(int));
    if (!sort_sample) {
        printf("Memory allocation failed during sort\n");
        free(data);
        return;
    }

    for (size_t i = 0; i < n; ++i) sort_sample[i] = data[i];
    clock_t start = clock();
    qsort(sort_sample, n, sizeof(int), cmpfunc);
    clock_t end = clock();
    printf(" | Sort: %.6f s\n", (double)(end - start) / CLOCKS_PER_SEC);
}

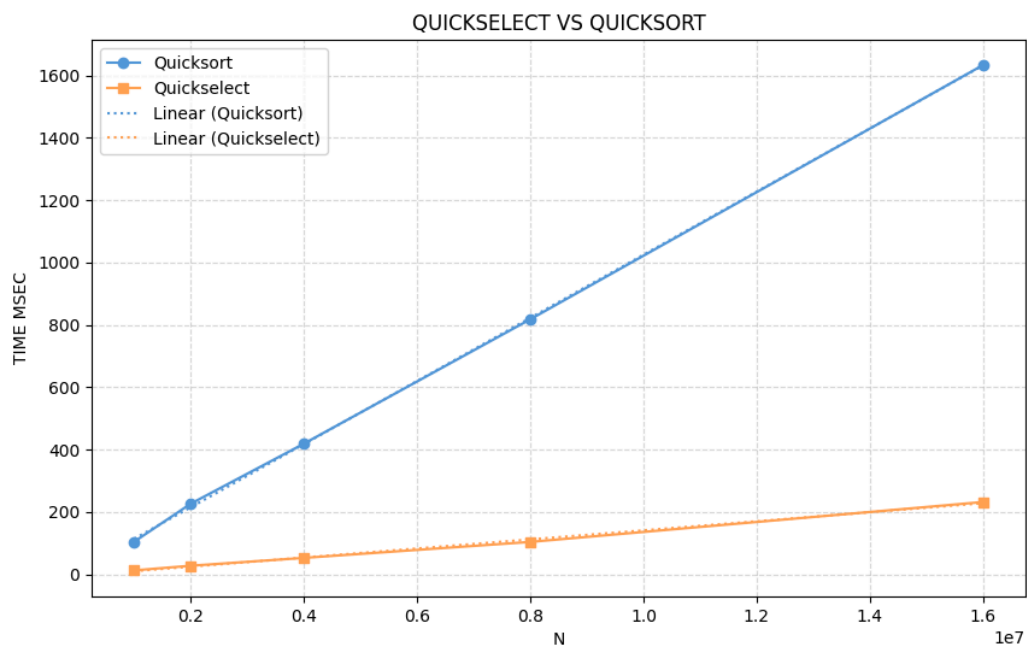
```

```

    free(data);
    free(sort_sample);
}

int main() {
    srand(time(NULL));
    size_t ns[] = {1000000, 2000000, 4000000, 8000000, 16000000};
    size_t len = sizeof(ns) / sizeof(ns[0]);
    for (size_t i = 0; i < len; ++i) {
        printf("n = %zu...\n", ns[i]);
        benchmark(ns[i]);
    }
    return 0;
}

```



Энэхүү графикаас харахад QuickSelect алгоритм нь Эрэмдэлэлт (QuickSort)-тэй харьцуулахад илүү хурдан ажиллаж байна. Оролтын хэмжээ (n) ихсэх үед хоёр аргын дундаж хугацааны бага зэрэг ихэссэн байна. QuickSelect-ийн дундаж хугацаа бага байх бол Эрэмдэлтийн хугацаа илүү хурдан өсөн нэмэгдэж байна.

2. Randomized BST болон Skip List өгөгдлийн бүтцүүдийн хайх үйлдэл зангилааны тоо $n = \{5M, 10M, 20M, 50M\}$ үед дунджаар тус бүр хэдэн зангилаа дамжиж буйг тоол. Ингэхдээ зангилаануудыг санамсаргүй үүсгэж нэг нэгээр нэмэх замаар өгөгдлийн бүтцээ үүсгэх ба уг бүтцээс санамсаргүй утга хайх үйлдлийг 1M удаа (байдаг утга 50%, байхгүй утга 50%) давтаж дамжсан зангилааны тооны дундажийг нь авна. Үр дүнгээр нь график эсвэл хүснэгт байгуул.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define MAX_LEVEL 16
#define NUM_SEARCHES 1000000

typedef struct SLNode {
    int key;
    struct SLNode **forward;
} SLNode;

typedef struct SkipList {
    SLNode *header;
    int level;
} SkipList;

int random_level() {
    int lvl = 1;
    while ((rand() & 1) && lvl < MAX_LEVEL) lvl++;
    return lvl;
}

SLNode* sl_new_node(int key, int level) {
    SLNode* n = malloc(sizeof(SLNode));
    n->key = key;
    n->forward = malloc(sizeof(SLNode*) * level);
    for (int i = 0; i < level; i++) n->forward[i] = NULL;
    return n;
}

SkipList* sl_create() {
    SkipList* sl = malloc(sizeof(SkipList));
    sl->level = 1;
```

```

    sl->header = sl_new_node(-1, MAX_LEVEL);
    return sl;
}

void sl_insert(SkipList* sl, int key) {
    SLNode* update[MAX_LEVEL];
    SLNode* x = sl->header;
    for (int i = sl->level - 1; i >= 0; i--) {
        while (x->forward[i] && x->forward[i]->key < key)
            x = x->forward[i];
        update[i] = x;
    }
    int lvl = random_level();
    if (lvl > sl->level) {
        for (int i = sl->level; i < lvl; i++)
            update[i] = sl->header;
        sl->level = lvl;
    }
    x = sl_new_node(key, lvl);
    for (int i = 0; i < lvl; i++)
        x->forward[i] = update[i]->forward[i], update[i]->forward[i] = x;
}

int sl_search(SkipList* sl, int key, int* steps) {
    SLNode* x = sl->header;
    *steps = 0;
    for (int i = sl->level - 1; i >= 0; i--) {
        while (x->forward[i] && x->forward[i]->key < key) {
            x = x->forward[i];
            (*steps)++;
        }
        (*steps)++;
    }
    x = x->forward[0];
    return (x && x->key == key);
}

typedef struct BSTNode {
    int key;
    struct BSTNode* left;
    struct BSTNode* right;
} BSTNode;

```

```

BSTNode* bst_new_node(int key) {
    BSTNode* node = malloc(sizeof(BSTNode));
    node->key = key;
    node->left = node->right = NULL;
    return node;
}

BSTNode* bst_insert(BSTNode* root, int key) {
    if (!root) return bst_new_node(key);
    if (key < root->key) root->left = bst_insert(root->left, key);
    else root->right = bst_insert(root->right, key);
    return root;
}

int bst_search(BSTNode* root, int key, int* steps) {
    *steps = 0;
    while (root) {
        (*steps)++;
        if (key == root->key) return 1;
        else if (key < root->key) root = root->left;
        else root = root->right;
    }
    return 0;
}

void benchmark(size_t n) {
    printf("== n = %zu ==\n", n);

    SkipList* sl = sl_create();
    BSTNode* bst_root = NULL;

    int* keys = malloc(n * sizeof(int));
    for (size_t i = 0; i < n; ++i) keys[i] = rand();

    for (size_t i = 0; i < n; ++i) {
        sl_insert(sl, keys[i]);
        bst_root = bst_insert(bst_root, keys[i]);
    }

    long long sl_total_steps = 0, bst_total_steps = 0;
    for (size_t i = 0; i < NUM_SEARCHES; ++i) {
        int target;
        if (rand() % 2 == 0)

```

```

        target = keys[rand() % n];
    else
        target = rand();

    int steps;
    sl_search(sl, target, &steps);
    sl_total_steps += steps;

    bst_search(bst_root, target, &steps);
    bst_total_steps += steps;
}

printf("Skip List:    Avg steps: %.2f\n", (double)sl_total_steps /
NUM_SEARCHES);
printf("Randomized BST: Avg steps: %.2f\n", (double)bst_total_steps /
NUM_SEARCHES);

free(keys);
}

```

```

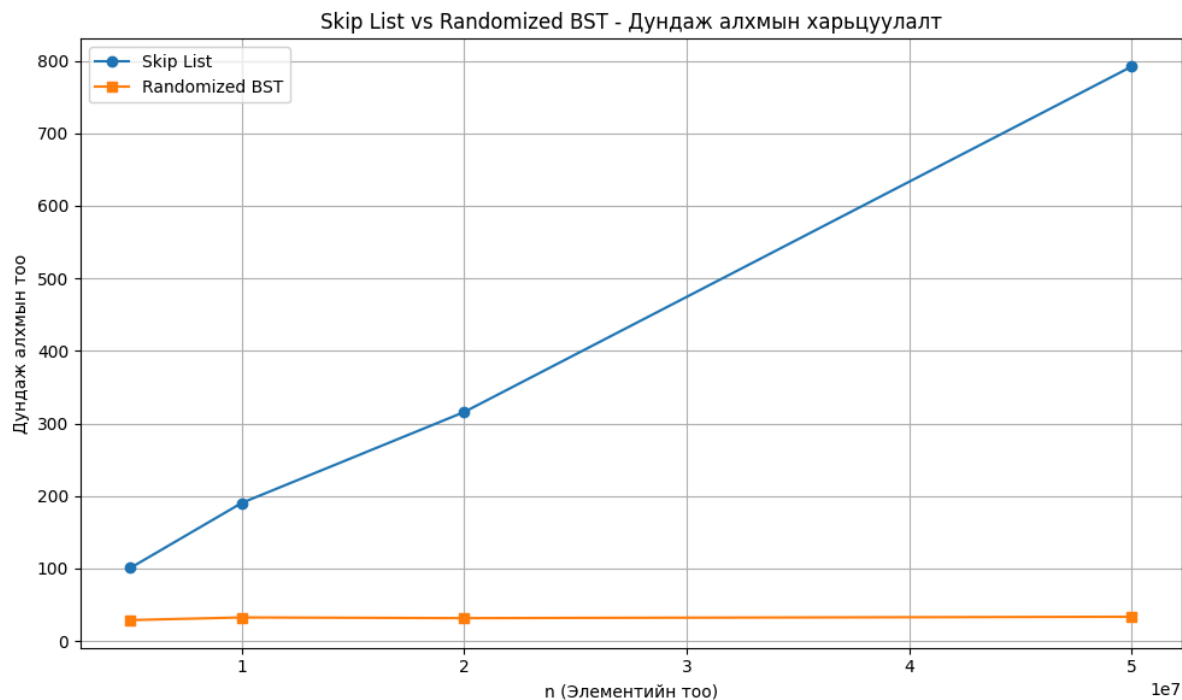
int main() {
    srand(time(NULL));
    size_t sizes[] = {5000000, 10000000, 20000000, 50000000};
    for (int i = 0; i < 4; ++i) {
        benchmark(sizes[i]);
    }
    return 0;
}

```

```

== n = 5000000 ==
Skip List:    Avg steps: 100.97
Randomized BST: Avg steps: 28.95
== n = 10000000 ==
Skip List:    Avg steps: 190.45
Randomized BST: Avg steps: 32.63
== n = 20000000 ==
Skip List:    Avg steps: 315.82
Randomized BST: Avg steps: 31.65
== n = 50000000 ==
Skip List:    Avg steps: 792.10
Randomized BST: Avg steps: 33.55

```

Энэ графикаас харахад "Randomized BST" (Санамсаргүй хоёртын хайлтын мод) нь "Skip List"-тэй харьцуулахад илүү үр дүнтэй байна. Учир нь элементийн тоо (n) ихэсэх тусам "Skip List"-ийн дундаж алхмын тоо маш хурдан өсч байгаа бол "Randomized BST"-ийн дундаж алхмын тоо тогтвортой, бага хэвээр байна. Иймд их хэмжээний өгөгдөлтэй үед "Randomized BST" хайлтын хувьд илүү хурдан, ашиглахад тохиромжтой болохыг харуулж байна.

github link: <https://github.com/Belegsee-Enkhbat/random-algo-report.git>